

## **LAPORAN PRAKTIKUM**

### **TUGAS 4 Web Service Container: Apache, Nginx, Reverse Proxy, dan TLS**

Dosen Pengampu : Ferry Astika Saputra



Disusun Oleh :

Adam Rasyid Nurmuhammad

3126640025

D4-IT-B

**PROGRAM STUDI SARJANA TERAPAN  
JURUSAN TEKNIK INFORMATIKA  
DEPARTEMEN TEKNIK INFORMATIKA DAN KOMPUTER  
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA  
2026**

# DAFTAR ISI

|                                                                               |           |
|-------------------------------------------------------------------------------|-----------|
| <b>DAFTAR ISI.....</b>                                                        | <b>2</b>  |
| <b>BAB I PENDAHULUAN.....</b>                                                 | <b>3</b>  |
| 1.1 Latar Belakang .....                                                      | 3         |
| 1.2 Tujuan Pembelajaran dan Kompetensi Utama .....                            | 3         |
| <b>BAB II LANDASAN TEORI .....</b>                                            | <b>4</b>  |
| 2.1 Apache httpd vs Nginx.....                                                | 4         |
| 2.2 Peta Konsep: TLS, Penyimpanan Private Key, dan TLS Offloading .....       | 4         |
| 2.3 Peta Konsep: Cara Kerja Reverse Proxy, Routing, dan Filter Keamanan ..... | 5         |
| <b>BAB III Pengerjaan Praktikum.....</b>                                      | <b>6</b>  |
| 3.1 Penyiapan Direktori Kerja dan Struktur Proyek .....                       | 6         |
| 3.2 Pembuatan Sertifikat TLS Laboratorium (Self-Signed) .....                 | 6         |
| 3.3 Konfigurasi Orkestrasi compose.yaml.....                                  | 7         |
| 3.4 Konfigurasi Reverse Proxy Nginx (default.conf) .....                      | 8         |
| 3.5 Pembuatan Dokumen Web Statis Apache (index.html) .....                    | 9         |
| 3.6 Penyiapan Dependensi Backend Flask (requirements.txt).....                | 10        |
| 3.7 Implementasi Kode Sumber Backend API (app.py) .....                       | 10        |
| 3.8 Pembuatan Dockerfile Kustom Backend Flask .....                           | 10        |
| 3.9 Validasi Konfigurasi dan Pemeriksaan Service .....                        | 11        |
| 3.10 Pembangunan Image dan Deployment Stack Compose .....                     | 12        |
| 3.11 Pengujian Cepat Fungsional Web Ingress dan API.....                      | 12        |
| 3.12 Pemeriksaan Boundary dan Isolasi Jaringan.....                           | 14        |
| 3.13 Pemeriksaan dan Audit Log Web Service.....                               | 16        |
| <b>BAB IV HASIL DAN PEMBAHASAN.....</b>                                       | <b>18</b> |
| 4.1 Evaluasi dan Latihan Mandiri .....                                        | 18        |
| 4.2 Analisis Wajib .....                                                      | 18        |
| <b>BAB V PENUTUP.....</b>                                                     | <b>20</b> |
| 5.1 Kesimpulan .....                                                          | 20        |

# **BAB I**

## **PENDAHULUAN**

### **1.1 Latar Belakang**

Dalam era arsitektur mikroservis dan praktik DevSecOps modern, web service berfungsi sebagai ingress boundary terdepan yang menghubungkan pengguna eksternal dengan komponen aplikasi dan basis data internal. Penerapan containerization memastikan lingkungan web server portabel, terisolasi, dan konsisten. Dalam arsitektur multi-tier yang aman, server aplikasi tidak boleh diekspos langsung ke publik. Sebaliknya, diterapkan pola Reverse Proxy (menggunakan Nginx) yang bertindak sebagai gerbang tunggal untuk menangani terminasi TLS/HTTPS (TLS Offloading), Layer-7 routing berbasis path (/ ke Apache dan /api/ ke Flask backend), serta penambahan filter keamanan HTTP security headers. Seluruh service backend diisolasi dalam private bridge network tanpa published port pada host, mewujudkan prinsip Least Exposure dan pertahanan berlapis (defense-in-depth).

### **1.2 Tujuan Pembelajaran dan Kompetensi Utama**

1. Mengidentifikasi attack surface umum Docker: root container, exposed socket, hardcoded secret, image rentan, dan resource abuse.
2. Menerapkan non-root user, read-only filesystem, drop capabilities, dan resource limits.
3. Memindai image dengan Trivy untuk menemukan CVE dan misconfiguration.
4. Membangun private registry untuk push/pull image internal.
5. Memahami posisi Docker Bench for Security sebagai audit baseline host.

## BAB II

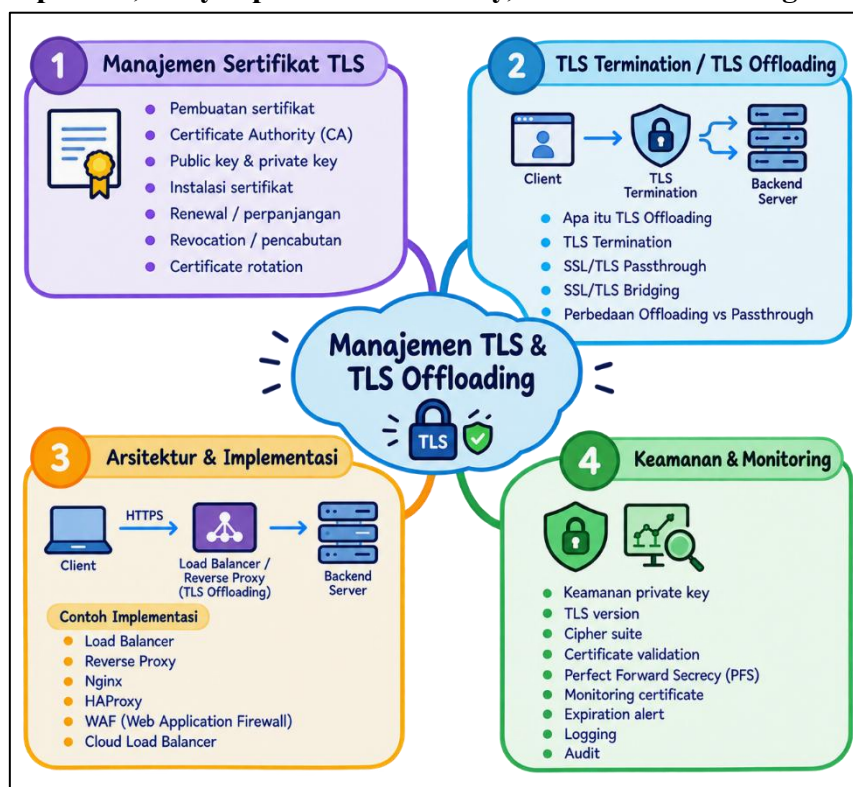
### LANDASAN TEORI

#### 2.1 Apache httpd vs Nginx

Apache HTTP Server menggunakan model Multi-Processing Modules (MPM) berbasis thread/proses per koneksi, sangat kaya fitur dengan modul dinamis dan dukungan file .htaccess, namun memiliki overhead memori tinggi pada konkurensi masif. Sebaliknya, Nginx mengadopsi arsitektur event-driven asynchronous non-blocking (epoll loop), sangat efisien dengan penggunaan resource CPU dan RAM yang konstan pada tingkat konkurensi tinggi, menjadikannya standar ideal untuk reverse proxy, static file serving, dan TLS termination.

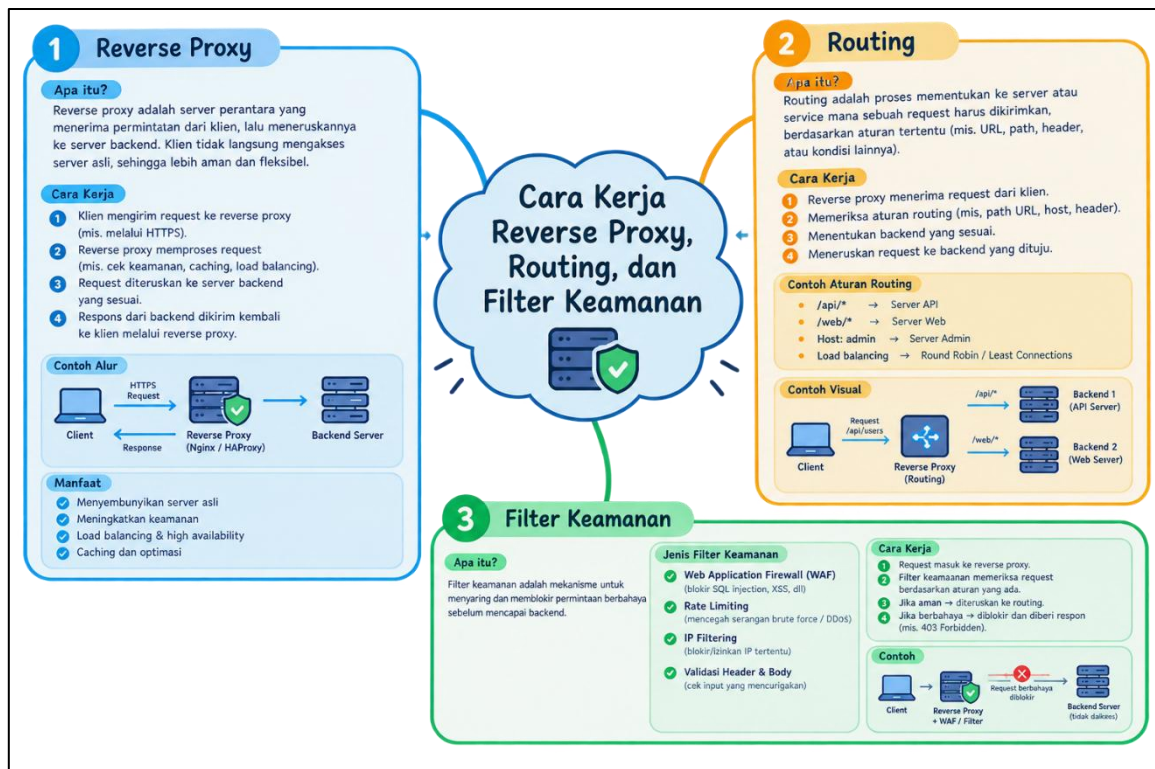
| Aspek Komparasi            | Apache HTTP Server (httpd)             | Nginx                                      |
|----------------------------|----------------------------------------|--------------------------------------------|
| Model Arsitektur           | Process/Thread per request (MPM)       | Asynchronous Event-driven loop (epoll)     |
| Konsumsi Resource          | Meningkat linier saat beban tinggi     | Sangat rendah dan stabil pada ribuan klien |
| Direktori Konfigurasi      | /usr/local/apache2/conf                | /etc/nginx                                 |
| Document Root Bawaan       | /usr/local/apache2/htdocs              | /usr/share/nginx/html                      |
| Konfigurasi Desentralisasi | Didukung via .htaccess                 | Tidak didukung (konfigurasi wajib sentral) |
| Fitur Reverse Proxy        | Memerlukan mod_proxy terpisah          | Kemampuan bawaan terintegrasi (proxy_pass) |
| Kasus Penggunaan Utama     | Aplikasi monolitik legacy, PHP mod_php | Reverse proxy, API gateway, TLS ingress    |

#### 2.2 Peta Konsep: TLS, Penyimpanan Private Key, dan TLS Offloading



Transport Layer Security (TLS) menjamin kerahasiaan data (enkripsi simetris AES-GCM), integritas data (hash SHA-256), dan autentikasi identitas server (sertifikat X.509). Konsep TLS Offloading (terminasi TLS) memusatkan proses dekripsi kriptografis pada reverse proxy Nginx sehingga beban komputasi server backend diringankan dan sertifikat dikelola terpusat di satu titik masuk. Terkait penyimpanan Private Key: kunci privat (lab.key) tidak boleh dimasukkan ke dalam image atau repositori Git. Kunci harus disuntikkan secara dinamis saat runtime melalui read-only bind mount (:ro) dengan izin berkas ketat (chmod 600) untuk meminimalkan risiko pencurian kunci privat.

## 2.3 Peta Konsep: Cara Kerja Reverse Proxy, Routing, dan Filter Keamanan



Reverse Proxy bertindak sebagai perantara yang mewakili server origin backend di hadapan klien eksternal. Nginx mengevaluasi URI request untuk melakukan Layer-7 Path Routing: rute root / diteruskan ke Apache web server, sedangkan rute /api/ dialihkan ke backend Flask. Proxy meneruskan header klien terpercaya (Host, X-Real-IP, X-Forwarded-For, X-Forwarded-Proto) untuk menjaga konteks alamat asli klien. Filter Keamanan: Nginx menyuntikkan header X-Content-Type-Options: nosniff (mencegah MIME sniffing), X-Frame-Options: DENY (mencegah Clickjacking), Referrer-Policy: no-referrer, serta pengalihan otomatis HTTP 301 ke HTTPS port 8443.

## BAB III

### Pengerjaan Praktikum

Struktur arsitektur berkas yang dibangun pada direktori kerja Bab 4 adalah sebagai berikut:

```
bab-4/
├── compose.yaml
├── nginx/
│   └── conf/
│       └── default.conf
├── apache/
│   └── sites/
│       └── index.html
├── app/
│   ├── Dockerfile
│   ├── requirements.txt
│   └── app.py
├── certs/
│   ├── lab.crt
│   └── lab.key
└── logs/
    └── nginx/
```

#### 3.1 Penyiapan Direktori Kerja dan Struktur Proyek

Membuat direktori kerja terisolasi `~/docker-lab/bab-4/{apache/sites,nginx/conf,certs,logs/nginx,app}` untuk mengorganisasi file konfigurasi, sertifikat, dan kode aplikasi.

```
adam@Devsecops:~$ mkdir -p ~/docker-lab/bab-4/{apache/sites,nginx/conf,certs,logs/nginx,app}
adam@Devsecops:~$ cd ~/docker-lab/bab-4
adam@Devsecops:~/docker-lab/bab-4$ ls
apache  app  certs  logs  nginx
```

*Gambar 3.1: Pembuatan Struktur Direktori Proyek Bab 4*

#### 3.2 Pembuatan Sertifikat TLS Laboratorium (Self-Signed)

Membangkitkan pasangan kunci privat RSA 2048-bit dan sertifikat x509 dengan ekstensi SAN untuk localhost dan 127.0.0.1 via OpenSSL, serta mengunci permission file dengan `chmod 600 certs/lab.key`.



```
GNU nano 6.2
services:
  proxy:
    image: nginx:alpine
    ports:
      - "127.0.0.1:8080:80"
      - "127.0.0.1:8443:443"
    volumes:
      - ./nginx/conf:/etc/nginx/conf.d:ro
      - ./certs:/etc/nginx/certs:ro
      - ./logs/nginx:/var/log/nginx
    networks:
      - web-net
    depends_on:
      apache-web:
        condition: service_started
      flask-app:
        condition: service_healthy

  apache-web:
    image: httpd:2.4-alpine
    volumes:
      - ./apache/sites:/usr/local/apache2/htdocs:ro
    networks:
      - web-net

  flask-app:
    build:
      context: ./app
    networks:
      - web-net
    healthcheck:
      test:
        - CMD
        - python
```

*Gambar 3.3: Deklarasi Layanan Multi-Container pada compose.yaml*

### 3.4 Konfigurasi Reverse Proxy Nginx (default.conf)

Menulis `nginx/conf/default.conf` untuk mendefinisikan upstream, redirect HTTP port 80 ke HTTPS 8443, SSL termination port 443, security headers, dan routing L7 ke Apache (/) dan Flask (/api/).

```

GNU nano 6.2                                                                    ngi
upstream apache_backend {
    server apache-web:80;
}

upstream flask_backend {
    server flask-app:5000;
}

server {
    listen 80;
    server_name localhost;
    return 301 https://$host:8443$request_uri;
}

server {
    listen 443 ssl;
    server_name localhost;

    ssl_certificate      /etc/nginx/certs/lab.crt;
    ssl_certificate_key  /etc/nginx/certs/lab.key;
    ssl_protocols TLSv1.2 TLSv1.3;

    add_header X-Content-Type-Options "nosniff" always;
    add_header X-Frame-Options "DENY" always;
    add_header Referrer-Policy "no-referrer" always;

    location / {
        proxy_pass http://apache_backend;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}

```

*Gambar 3.4: Konfigurasi Reverse Proxy, TLS, dan Routing Nginx*

### 3.5 Pembuatan Dokumen Web Statis Apache (index.html)

Membuat dokumen HTML pada `apache/sites/index.html` yang memuat teks konfirmasi 'Apache di Belakang Nginx' untuk disajikan oleh origin server Apache.

```

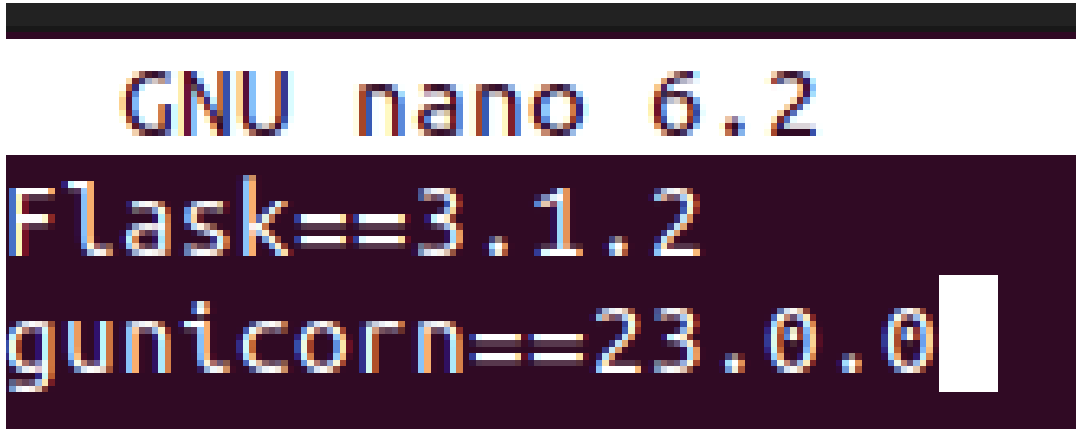
GNU nano 6.2
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <title>DevSecOps Bab 4</title>
</head>
<body>
    <h1>Apache di Belakang Nginx</h1>
    <p>Halaman Bab 4 berhasil dilayani melalui reverse proxy.</p>
    <p><a href="/api/">Uji Flask API</a></p>
</body>
</html>

```

*Gambar 3.5: Pembuatan File index.html untuk Apache Web*

### 3.6 Penyiapan Dependensi Backend Flask (requirements.txt)

Mendefinisikan library Flask==3.1.2 dan server WSGI gunicorn==23.0.0 untuk mendukung eksekusi backend Python yang handal.



```
GNU nano 6.2
Flask==3.1.2
gunicorn==23.0.0
```

*Gambar 3.6: Definisi Dependensi Python pada requirements.txt*

### 3.7 Implementasi Kode Sumber Backend API (app.py)

Menulis kode backend Python yang menyediakan route / (mengembalikan status ok dan header X-Forwarded-Proto) serta route /health untuk mekanisme healthcheck.



```
GNU nano 6.2
from flask import Flask, jsonify, request

app = Flask(__name__)

@app.get("/")
def index():
    return jsonify(
        status="ok",
        service="flask-app",
        message="API Bab 4 berhasil diakses melalui Nginx",
        forwarded_proto=request.headers.get("X-Forwarded-Proto"),
    )

@app.get("/health")
def health():
    return jsonify(status="healthy"), 200
```

*Gambar 3.7: Implementasi Endpoint API Flask pada app.py*

### 3.8 Pembuatan Dockerfile Kustom Backend Flask

Menyusun Dockerfile berbasis python:3.12-slim, menginstal dependensi, membuat akun sistem appuser (UID 10001), dan mengeksekusi Gunicorn dengan 2 worker.

```

GNU nano 6.2 app/Dockerfile
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

RUN useradd --system --uid 10001 --no-create-home appuser
USER appuser

EXPOSE 5000

CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--workers", "2", "app:app"]

```

Gambar 3.8: Dockerfile Backend dengan Hardening Akun Non-Root

### 3.9 Validasi Konfigurasi dan Pemeriksaan Service

Memeriksa pohon direktori dengan `find .` dan memvalidasi model konfigurasi menggunakan `docker compose config` untuk memastikan tidak ada kesalahan sintaks YAML.

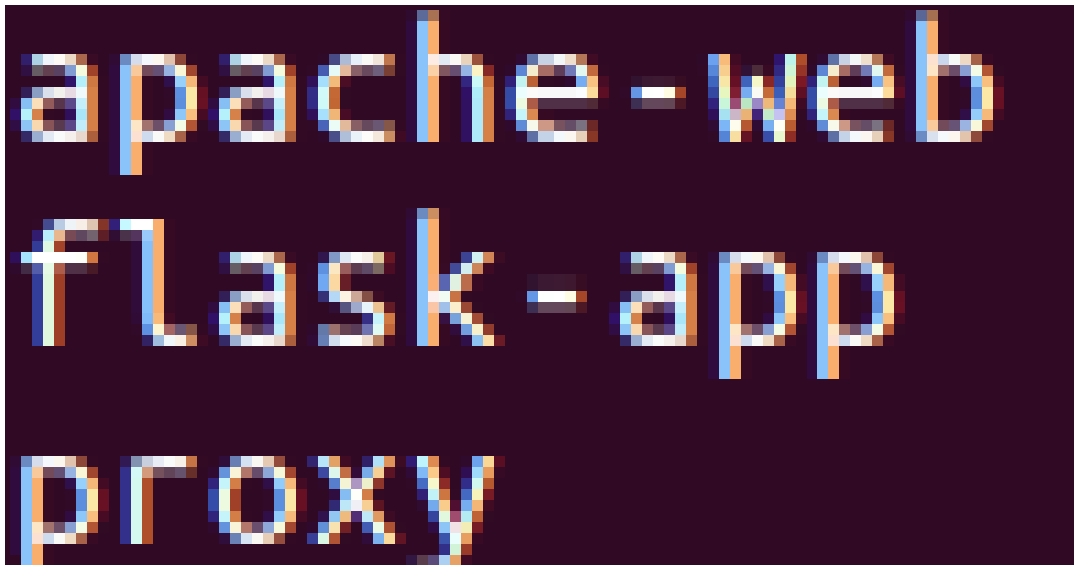
```

adam@Devsecops:~/docker-lab/bab-4$ cd ~/docker-lab/bab-4
find . -maxdepth 3 -type f | sort
docker compose config
docker compose config --services
./apache/sites/index.html
./app/app.py
./app/Dockerfile
./app/requirements.txt
./certs/lab.crt
./certs/lab.key
./compose.yaml
./nginx/conf/default.conf
name: bab-4
services:
  apache-web:
    image: httpd:2.4-alpine
    networks:
      web-net: null
    volumes:
      - type: bind
        source: /home/adam/docker-lab/bab-4/apache/sites
        target: /usr/local/apache2/htdocs
        read_only: true
        bind: {}
  flask-app:
    build:
      context: /home/adam/docker-lab/bab-4/app
      dockerfile: Dockerfile
    healthcheck:
      test:

```

Gambar 3.9: Validasi Hierarki File dan Sintaks Docker Compose

Memverifikasi daftar service yang berhasil dikenali oleh Docker Compose: proxy, apache-web, dan flask-app.



Gambar 3.10: Pemeriksaan Daftar Service Terdaftar

### 3.10 Pembangunan Image dan Deployment Stack Compose

Mengeksekusi `docker compose up -d --build` untuk membangun image backend dan menjalankan seluruh container secara terorkestrasi.

```
adam@Devsecops:~/docker-lab/bab-4$ docker compose up -d --build
docker compose ps
docker compose logs --tail 100
```

Gambar 3.11: Proses Build dan Eksekusi Stack di Background

Cuplikan log konsolidasi yang membuktikan Apache 2.4.68 aktif, entrypoint Nginx siap, dan Unicorn worker berhasil booting mendengarkan port 5000.

```
apache-web-1 | AH00558: httpd: Could not reliably determine the server's fully qualified domain name, using 172.21.0.3. See
e globally to suppress this message
apache-web-1 | AH00558: httpd: Could not reliably determine the server's fully qualified domain name, using 172.21.0.3. See
e globally to suppress this message
apache-web-1 | [Thu Oct 01 03:40:53.898338 2026] [mpm_event:notice] [pid 1:tid 1] AH00489: Apache/2.4.68 (Unix) configured
ons
apache-web-1 | [Thu Oct 01 03:40:53.898696 2026] [core:notice] [pid 1:tid 1] AH00094: Command line: 'httpd -D FOREGROUND'
proxy-1 | /docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
proxy-1 | /docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
proxy-1 | /docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: can not modify /etc/nginx/conf.d/default.conf (read-only file system)
proxy-1 | /docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
proxy-1 | /docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
proxy-1 | /docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
proxy-1 | /docker-entrypoint.sh: Configuration complete; ready for start up
flask-app-1 | [2026-10-01 03:40:53 +0000] [1] [INFO] Starting gunicorn 23.0.0
flask-app-1 | [2026-10-01 03:40:53 +0000] [1] [INFO] Listening at: http://0.0.0.0:5000 (1)
flask-app-1 | [2026-10-01 03:40:53 +0000] [1] [INFO] Using worker: sync
flask-app-1 | [2026-10-01 03:40:53 +0000] [7] [INFO] Booting worker with pid: 7
flask-app-1 | [2026-10-01 03:40:53 +0000] [8] [INFO] Booting worker with pid: 8
```

Gambar 3.12: Log Konsolidasi Startup Service

### 3.11 Pengujian Cepat Fungsional Web Ingress dan API

Eksekusi `curl -I http://localhost:8080/` menghasilkan respon HTTP/1.1 301 Moved Permanently yang secara otomatis mengarahkan klien ke `https://localhost:8443/`.

```
adam@Devsecops:~/docker-lab/bab-4$ curl -I http://localhost:8080/
HTTP/1.1 301 Moved Permanently
Server: nginx/1.31.5
Date: Thu, 01 Oct 2026 03:47:39 GMT
Content-Type: text/html
Content-Length: 169
Connection: keep-alive
Location: https://localhost:8443/
```

Gambar 3.13: Pengujian HTTP ke HTTPS Auto-Redirect (Port 8080)

Eksekusi `curl -k -i https://localhost:8443/` menghasilkan respon 200 OK dengan security headers (nosniff, DENY, no-referrer) dan menyajikan konten index.html Apache.

```
adam@Devsecops:~/docker-lab/bab-4$ curl -k -i https://localhost:8443/
HTTP/1.1 200 OK
Server: nginx/1.31.5
Date: Thu, 01 Oct 2026 03:48:18 GMT
Content-Type: text/html
Content-Length: 282
Connection: keep-alive
Last-Modified: Thu, 01 Oct 2026 03:36:17 GMT
ETag: "11a-65cbf1e56bcd7"
Accept-Ranges: bytes
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Referrer-Policy: no-referrer

<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>DevSecOps Bab 4</title>
</head>
<body>
  <h1>Apache di Belakang Nginx</h1>
  <p>Halaman Bab 4 berhasil dilayani melalui reverse proxy.</p>
  <p><a href="/api/">Uji Flask API</a></p>
</body>
</html>
```

Gambar 3.14: Pengujian Akses Konten Apache melalui Nginx (Port 8443)

Eksekusi `curl -k https://localhost:8443/api/` mengembalikan JSON dengan pesan konfirmasi dan membuktikan `forwarded_proto` berhasil terbaca sebagai https.

```
adam@Devsecops:~/docker-lab/bab-4$ curl -k https://localhost:8443/api/
{"forwarded_proto":"https","message":"API Bab 4 berhasil diakses melalui Nginx","service":"flask-app","status":"ok"}
```

Gambar 3.15: Pengujian Routing API Backend (/api/)

Eksekusi `curl -k -i https://localhost:8443/api/health` mengembalikan status HTTP 200 OK `{'status':'healthy'}` yang membuktikan service backend siap menerima beban kerja.

```
adam@Devsecops:~/docker-lab/bab-4$ curl -k -i https://localhost:8443/api/health
HTTP/1.1 200 OK
Server: nginx/1.31.5
Date: Thu, 01 Oct 2026 03:49:29 GMT
Content-Type: application/json
Content-Length: 21
Connection: keep-alive
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Referrer-Policy: no-referrer

{"status": "healthy"}
```

Gambar 3.16: Pengujian Health Endpoint Backend (/api/health)

Pengujian menggunakan openssl s\_client memvalidasi bahwa koneksi HTTPS berhasil dinegosiasikan menggunakan protokol mutakhir TLSv1.3 dan cipher suite TLS\_AES\_256\_GCM\_SHA384.

```
adam@Devsecops:~/docker-lab/bab-4$ openssl s_client \
  -connect localhost:8443 \
  -servername localhost \
  -brief </dev/null
depth=0 CN = localhost, O = DevSecOps Docker Lab
verify error:num=18:self-signed certificate
CONNECTION ESTABLISHED
Protocol version: TLSv1.3
Ciphersuite: TLS_AES_256_GCM_SHA384
Peer certificate: CN = localhost, O = DevSecOps Docker Lab
Hash used: SHA256
Signature type: RSA-PSS
Verification error: self-signed certificate
Server Temp Key: X25519, 253 bits
DONE
```

Gambar 3.17: Verifikasi Kriptografi Handshake TLS via OpenSSL

### 3.12 Pemeriksaan Boundary dan Isolasi Jaringan

Menjalankan perintah inspeksi status container, inspeksi bridge network, serta uji pemanggilan internal via wget dari dalam container proxy.

```
adam@Devsecops:~/docker-lab/bab-4$ docker compose ps
docker network inspect bab-4_web-net
docker compose exec proxy wget -qO- http://apache-web/
docker compose exec proxy wget -qO- http://flask-app:5000/health
```

Gambar 3.18: Perintah Uji Integritas Boundary Jaringan

Pemeriksaan docker compose ps membuktikan hanya proxy yang memiliki published port (8080 & 8443 terikat ke 127.0.0.1), sedangkan Apache dan Flask murni tertutup di internal network.

| NAME               | IMAGE            | COMMAND                   | SERVICE    | CREATED       | STATUS                 | PORTS                                           |
|--------------------|------------------|---------------------------|------------|---------------|------------------------|-------------------------------------------------|
| bab-4-apache-web-1 | httpd:2.4-alpine | "httpd-foreground"        | apache-web | 9 minutes ago | Up 9 minutes           | 80/tcp                                          |
| bab-4-flask-app-1  | bab-4-flask-app  | "gunicorn --bind 0.0.0.0" | flask-app  | 9 minutes ago | Up 9 minutes (healthy) | 5000/tcp                                        |
| bab-4-proxy-1      | nginx:alpine     | "/docker-entrypoint..."   | proxy      | 9 minutes ago | Up 9 minutes           | 127.0.0.1:8080->80/tcp, 127.0.0.1:8443->443/tcp |

Gambar 3.19: Verifikasi Pembatasan Published Port

Hasil docker network inspect bab-4\_web-net menunjukkan ketiga container terhubung dalam satu subnet bridge privat (172.21.0.0/16).

```
.1:8443->443/tcp
[
  {
    "Name": "bab-4_web-net",
    "Id": "4d0ea070447c1b4b78d70bd2e752fee5286fd2c71a5eb8d0f2a657a538635ee4",
    "Created": "2026-10-01T10:40:49.115084617+07:00",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv4": true,
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": null,
      "Config": [
        {
          "Subnet": "172.21.0.0/16",
          "Gateway": "172.21.0.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
    "Options": {},
    "Labels": {
      "com.docker.compose.config-hash": "9c7bc8f98208b81a954f52f24050cdef15c96f9a7c8240358775a5d53a025a90",
      "com.docker.compose.network": "web-net",
      "com.docker.compose.project": "bab-4"
    }
  }
]
```

Gambar 3.20: Inspeksi Alokasi IP pada Bridge Network web-net

Perintah `wget -qO- http://apache-web/` dari dalam proxy membuktikan komunikasi internal menggunakan nama service berjalan lancar.

```
com.docker.compose.project: bab-4
com.docker.compose.version: 1.29.1
},
"Containers": {
  "02a80bcb24cd286d5bfacfe0b521a7f66970e799b1d0d5d7579731d0d1e82cc8": {
    "Name": "bab-4-proxy-1",
    "EndpointID": "212b1d7cb4c3ba98380247919ded5e0abfa61dd93aaeac0fb5bb579511fea1b7",
    "MacAddress": "8e:3b:d4:27:e7:1d",
    "IPv4Address": "172.21.0.4/16",
    "IPv6Address": ""
  },
  "601823a26a13e8def372ddd05161749f5d200c053a5a73f55887bcc91a73a4bf": {
    "Name": "bab-4-apache-web-1",
    "EndpointID": "370f1bd6011c98453b1a419aa5b9745877e0cece338c4516a33b1ecf39736be1",
    "MacAddress": "fe:d3:41:5d:43:94",
    "IPv4Address": "172.21.0.3/16",
    "IPv6Address": ""
  },
  "d835d6837093a4b35ae9bbe6ccfd2e834bd6d53d72c031484f564f66ac09767d": {
    "Name": "bab-4-flask-app-1",
    "EndpointID": "182d2fd648323a97d71fe6986016f1fb4ca9520c2cab4623e5312250605c63bf",
    "MacAddress": "5e:f3:6b:5d:df:7f",
    "IPv4Address": "172.21.0.2/16",
    "IPv6Address": ""
  }
},
"Status": {
  "IPAM": {
    "Subnets": {
      "172.21.0.0/16": {
        "IPsInUse": 6,
        "DynamicIPsAvailable": 65530
      }
    }
  }
}
```

Gambar 3.21: Uji Konektivitas Internal Proxy ke Apache via DNS

Perintah `wget -qO- http://flask-app:5000/health` dari dalam proxy membuktikan resolusi nama internal ke backend Flask sukses.

```

    }
  }
}
]
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>DevSecOps Bab 4</title>
</head>
<body>
  <h1>Apache di Belakang Nginx</h1>
  <p>Halaman Bab 4 berhasil dilayani melalui reverse proxy.</p>
  <p><a href="/api/">Uji Flask API</a></p>
</body>
</html>
{"status": "healthy"}

```

*Gambar 3.22: Uji Konektivitas Internal Proxy ke Flask via DNS*

### 3.13 Pemeriksaan dan Audit Log Web Service

Mengeksekusi pembacaan log container via docker compose logs serta memeriksa langsung berkas log fisik pada bind mount host logs/nginx/.

```

adam@Devsecops:~/docker-lab/bab-4$ docker compose logs proxy
docker compose logs apache-web
docker compose logs flask-app
tail -n 20 logs/nginx/access.log
tail -n 20 logs/nginx/error.log

```

*Gambar 3.23: Perintah Pengecekan Log Akses dan Error*

Output log konsolidasi menampilkan catatan aktivitas request HTTP 301, GET /, GET /api/, dan GET /api/health secara transparan dan kronologis.

```

proxy-1 | /docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
proxy-1 | /docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
proxy-1 | /docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
proxy-1 | 10-listen-on-ipv6-by-default.sh: info: can not modify /etc/nginx/conf.d/default.conf (read-only file system)
proxy-1 | /docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
proxy-1 | /docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
proxy-1 | /docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
proxy-1 | /docker-entrypoint.sh: Configuration complete; ready for start up
apache-web-1 | AH00558: httpd: Could not reliably determine the server's fully qualified domain name, using 172.21.0.1
apache-web-1 | AH00558: httpd: Could not reliably determine the server's fully qualified domain name, using 172.21.0.1
apache-web-1 | [Thu Oct 01 03:40:53.898338 2026] [mpm_event:notice] [pid 1:tid 1] AH00489: Apache/2.4.68 (Ubuntu) configured
apache-web-1 | [Thu Oct 01 03:40:53.898696 2026] [core:notice] [pid 1:tid 1] AH00094: Command line: 'httpd -DFOREGROUND'
apache-web-1 | 172.21.0.4 - - [01/Oct/2026:03:48:18 +0000] "GET / HTTP/1.1" 200 282
apache-web-1 | 172.21.0.4 - - [01/Oct/2026:03:50:40 +0000] "GET / HTTP/1.1" 200 282
flask-app-1 | [2026-10-01 03:40:53 +0000] [1] [INFO] Starting gunicorn 23.0.0
flask-app-1 | [2026-10-01 03:40:53 +0000] [1] [INFO] Listening at: http://0.0.0.0:5000 (1)
flask-app-1 | [2026-10-01 03:40:53 +0000] [1] [INFO] Using worker: sync
flask-app-1 | [2026-10-01 03:40:53 +0000] [7] [INFO] Booting worker with pid: 7
flask-app-1 | [2026-10-01 03:40:53 +0000] [8] [INFO] Booting worker with pid: 8
172.21.0.1 - - [01/Oct/2026:03:47:39 +0000] "HEAD / HTTP/1.1" 301 0 "-" "curl/7.81.0" "-"
172.21.0.1 - - [01/Oct/2026:03:48:18 +0000] "GET / HTTP/1.1" 200 282 "-" "curl/7.81.0" "-"
172.21.0.1 - - [01/Oct/2026:03:48:53 +0000] "GET /api/ HTTP/1.1" 200 117 "-" "curl/7.81.0" "-"
172.21.0.1 - - [01/Oct/2026:03:49:29 +0000] "GET /api/health HTTP/1.1" 200 21 "-" "curl/7.81.0" "-"
2026/10/01 03:40:58 [notice] 1#1: using the "epoll" event method
2026/10/01 03:40:58 [notice] 1#1: nginx/1.31.5
2026/10/01 03:40:58 [notice] 1#1: built by gcc 15.2.0 (Alpine 15.2.0)
2026/10/01 03:40:58 [notice] 1#1: OS: Linux 6.8.0-138-generic
2026/10/01 03:40:58 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1024:524288
2026/10/01 03:40:58 [notice] 1#1: start worker processes
2026/10/01 03:40:58 [notice] 1#1: start worker process 21
2026/10/01 03:40:58 [notice] 1#1: start worker process 22
2026/10/01 03:40:58 [notice] 1#1: start worker process 23
2026/10/01 03:40:58 [notice] 1#1: start worker process 24

```

Gambar 3.24: Jejak Audit Request pada Nginx, Apache, dan Gunicorn

## **BAB IV**

### **HASIL DAN PEMBAHASAN**

#### **4.1 Evaluasi dan Latihan Mandiri**

##### **1. Mengapa reverse proxy tidak seharusnya menjalankan semua logic aplikasi?**

Reverse proxy dirancang menggunakan arsitektur event-driven non-blocking I/O yang optimal untuk menangani koneksi konkuren masif, terminasi TLS, kompresi, dan static caching. Membebaskan logika bisnis ke proxy melanggar prinsip Single Responsibility. Tugas CPU-bound atau blocking I/O pada proxy akan menahan worker process, menurunkan throughput server secara drastis bagi seluruh klien, serta memperluas attack surface (blast radius).

##### **2. Apa perbedaan TLS termination dan end-to-end TLS?**

Pada TLS termination, enkripsi TLS didekripsi di reverse proxy dan diteruskan sebagai HTTP biasa dalam private network internal; metode ini hemat CPU dan memusatkan sertifikat. Sedangkan pada end-to-end TLS, enkripsi dipertahankan hingga container backend origin (proxy melakukan pass-through atau re-encryption), wajib diterapkan pada arsitektur Zero-Trust meskipun menambah overhead komputasi ganda.

##### **3. Bagaimana cara mengisolasi backend agar tidak langsung diakses dari host?**

Dengan tidak menyertakan deklarasi ports: pada service backend (Apache dan Flask) di file compose.yaml, menempatkan backend dalam custom user-defined bridge network (web-net), dan hanya mengekspos port proxy yang dibatasi pada loopback host (127.0.0.1:8080 dan 127.0.0.1:8443).

##### **4. Apa konsekuensi menyimpan private key TLS di bind mount?**

Menyimpan private key di bind mount bergantung pada izin file host (wajib chmod 600). Jika bind mount tidak diset read-only (:ro), container yang disusupi dapat merusak kunci. Selain itu, terdapat risiko kunci privat tidak sengaja ter-commit ke repositori Git publik jika folder certs tidak di-exclude di .gitignore, serta tidak adanya fitur rotasi secret otomatis.

##### **5. Bandingkan log Nginx dan log Apache dari sisi format dan kegunaan debugging.**

Keduanya mendukung Combined Log Format. Nginx unggul dalam kustomisasi variabel metrik proxy (\$upstream\_response\_time, \$upstream\_status, \$request\_time) yang sangat krusial untuk melacak bottleneck backend. Apache memisahkan error log dengan kode error modul internal yang sangat deskriptif (misal: AH00558) untuk debugging modul server.

#### **4.2 Analisis Wajib**

##### **1. Masalah yang Muncul dan Metode Diagnosis:**

- **Pesan Info Read-Only pada Nginx:** Saat container proxy dijalankan, muncul log 10-listen-on-ipv6-by-default.sh: info: can not modify /etc/nginx/conf.d/default.conf (read-only file system). Diagnosis menunjukkan bahwa skrip inisialisasi Nginx mencoba mengubah default.conf untuk IPv6, namun ditolak kernel karena konfigurasi di-mount dengan opsi :ro demi alasan keamanan. Masalah ini tidak mengganggu operasional karena konfigurasi telah mendengarkan IPv4 secara eksplisit.

- **Routing Prefix Mismatch:** Pada konfigurasi location /api/, penggunaan trailing slash pada proxy\_pass http://flask\_backend/; sangat krusial. Jika tanda slash dihilangkan, Nginx akan meneruskan path lengkap /api/ ke Flask dan menyebabkan galat 404 Not Found karena kode app.py hanya menangani route root /.
- **Validasi Sertifikat Self-Signed:** Eksekusi cURL HTTPS standar menghasilkan SSL certificate problem: self-signed certificate. Penanganan dilakukan dengan menyertakan flag -k (insecure) saat pengujian lokal sembari memvalidasi keabsahan parameter cipher suite melalui OpenSSL s\_client.

## 2. Risiko Keamanan dan Operasional Relevan:

- **Sertifikat Self-Signed di Publik:** Membuka celah serangan Man-In-The-Middle (MITM) dan melatih pengguna mengabaikan peringatan keamanan browser.
- **Trafik Internal Plaintext:** Komunikasi Nginx ke backend berjalan via HTTP biasa; jika host atau network Docker disusupi, paket data dapat disadap.
- **Header Spoofing:** Jika backend dapat diakses langsung di luar proxy, penyerang dapat memalsukan header X-Forwarded-Proto atau X-Real-IP.
- **Information Leakage di Log:** Berkas access.log berisiko merekam query string sensitif jika API menerima token/kredensial melalui URL.
- **Ketiadaan Rate Limiting:** Ketiadaan pembatasan laju request membuat reverse proxy rentan mengalami overload akibat serangan HTTP Flood (DoS).

## 3. Rekomendasi Perbaikan untuk Production-Like Environment:

- **Otomatisasi Sertifikat Resmi:** Menerbitkan sertifikat TLS resmi menggunakan Let's Encrypt / ACME dengan otomatisasi renewal berkala tanpa downtime.
- **Mutual TLS (mTLS):** Menerapkan Mutual TLS (mTLS) antar-kontainer internal untuk mewujudkan arsitektur Zero-Trust.
- **Secret Manager Terpusat:** Memindahkan penyimpanan private key dari bind mount ke Docker Secrets atau HashiCorp Vault.
- **WAF & Rate Limiting:** Mengaktifkan Web Application Firewall (ModSecurity/Coraza WAF) dan menerapkan limit\_req\_zone pada Nginx untuk mitigasi DoS.
- **Hardening Container:** Menetapkan batasan resource cgroup (CPU dan RAM limits) pada file Compose serta menjalankan seluruh container dengan read\_only: true.

## **BAB V**

### **PENUTUP**

#### **5.1 Kesimpulan**

Praktikum Bab 4 mengenai Web Service Container (Apache, Nginx, Reverse Proxy, dan TLS) berhasil mengimplementasikan arsitektur multi-tier yang tangguh, modular, dan aman. Penerapan Nginx sebagai reverse proxy di baris terdepan terbukti efektif memisahkan antarmuka publik dari origin backend, memusatkan terminasi TLSv1.3 (TLS Offloading), dan mengarahkan lalu lintas data secara dinamis ke Apache (konten statis) dan Flask (API dinamis). Penegakan prinsip Least Exposure berhasil dicapai dengan membatasi port publishing hanya pada reverse proxy dan mengisolasi backend di dalam private bridge network. Selain itu, integrasi HTTP security headers, penguncian izin kunci privat, eksekusi backend dengan akun non-root, pemantauan kesiapan layanan via healthcheck, serta persistensi access log menyediakan fondasi operasional dan keamanan yang esensial dalam siklus hidup DevSecOps skala produksi.